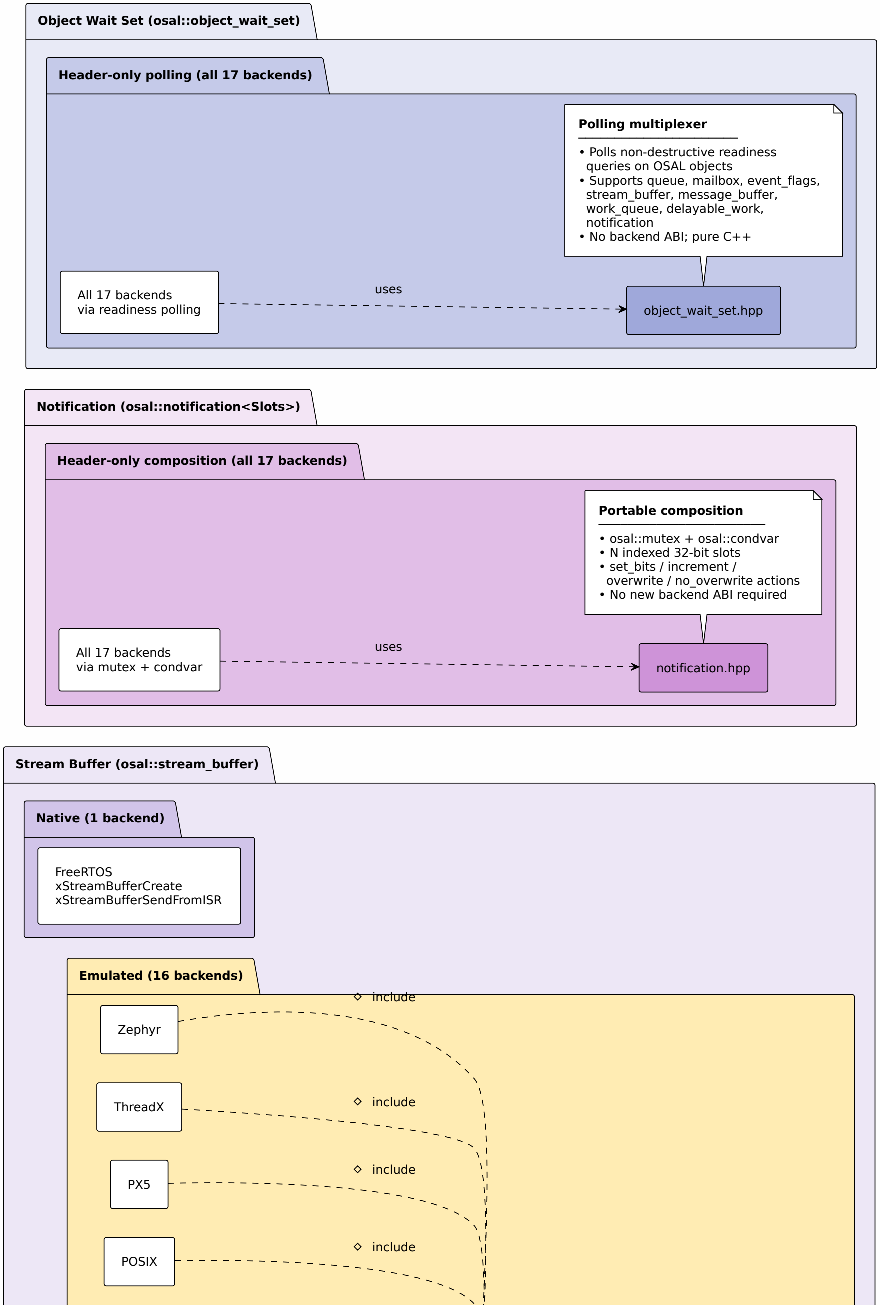
MicrOSAL — Native vs Emulated Primitives

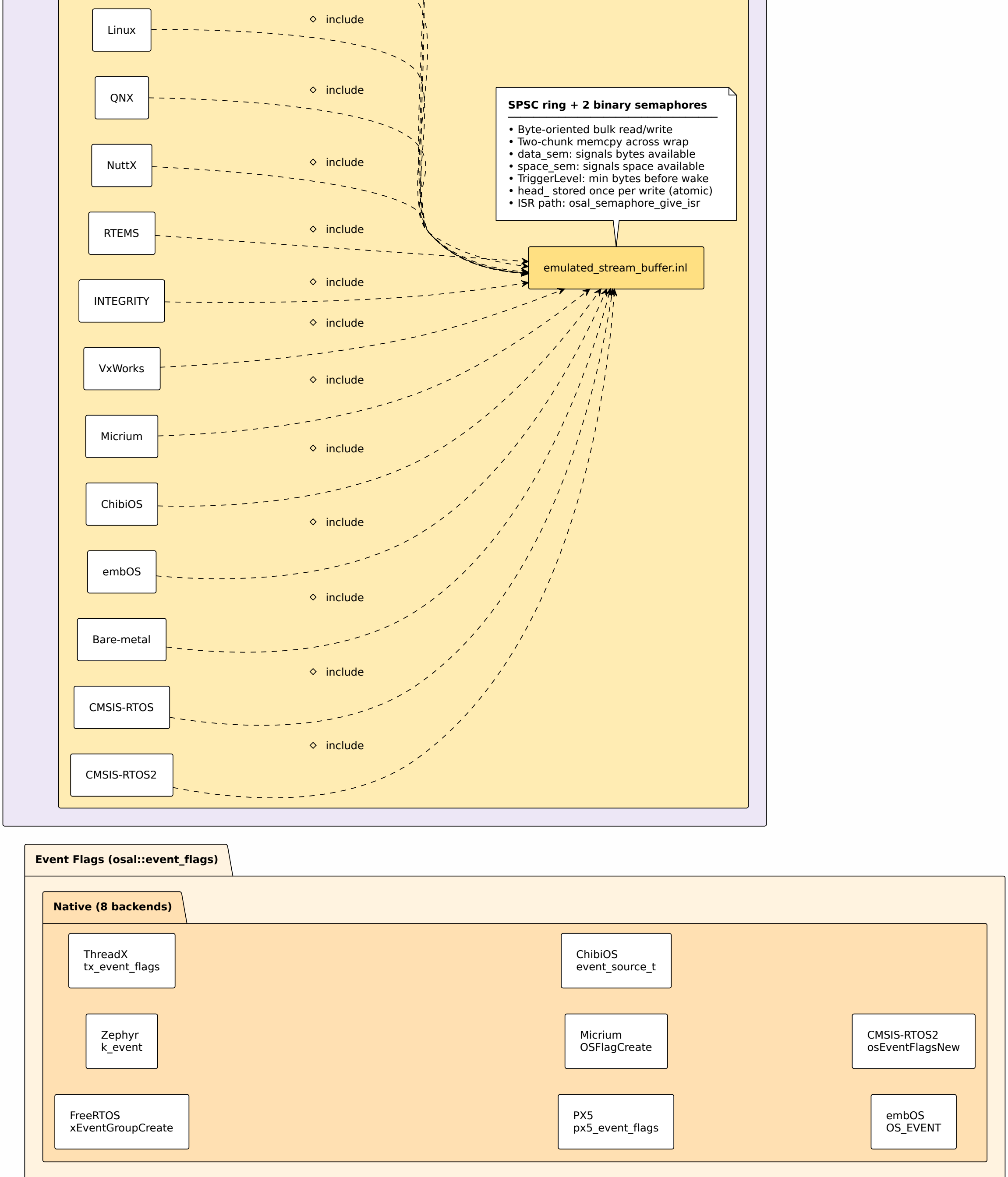

Object Wait Set (osal::object_wait_set)
Header-only polling (all 17 backends)
All 17 backends via readiness polling
uses
object_wait_set.hpp
Notification (osal::notification<Slots>)
Header-only composition (all 17 backends)
All 17 backends via mutex + condvar
uses
notification.hpp
Stream Buffer (osal::stream_buffer)
Native (1 backend)
FreeRTOS xStreamBufferCreate xStreamBufferSendFromISR
Emulated (16 backends)
Zephyr
include
ThreadX
include
PX5
include
POSIX
include

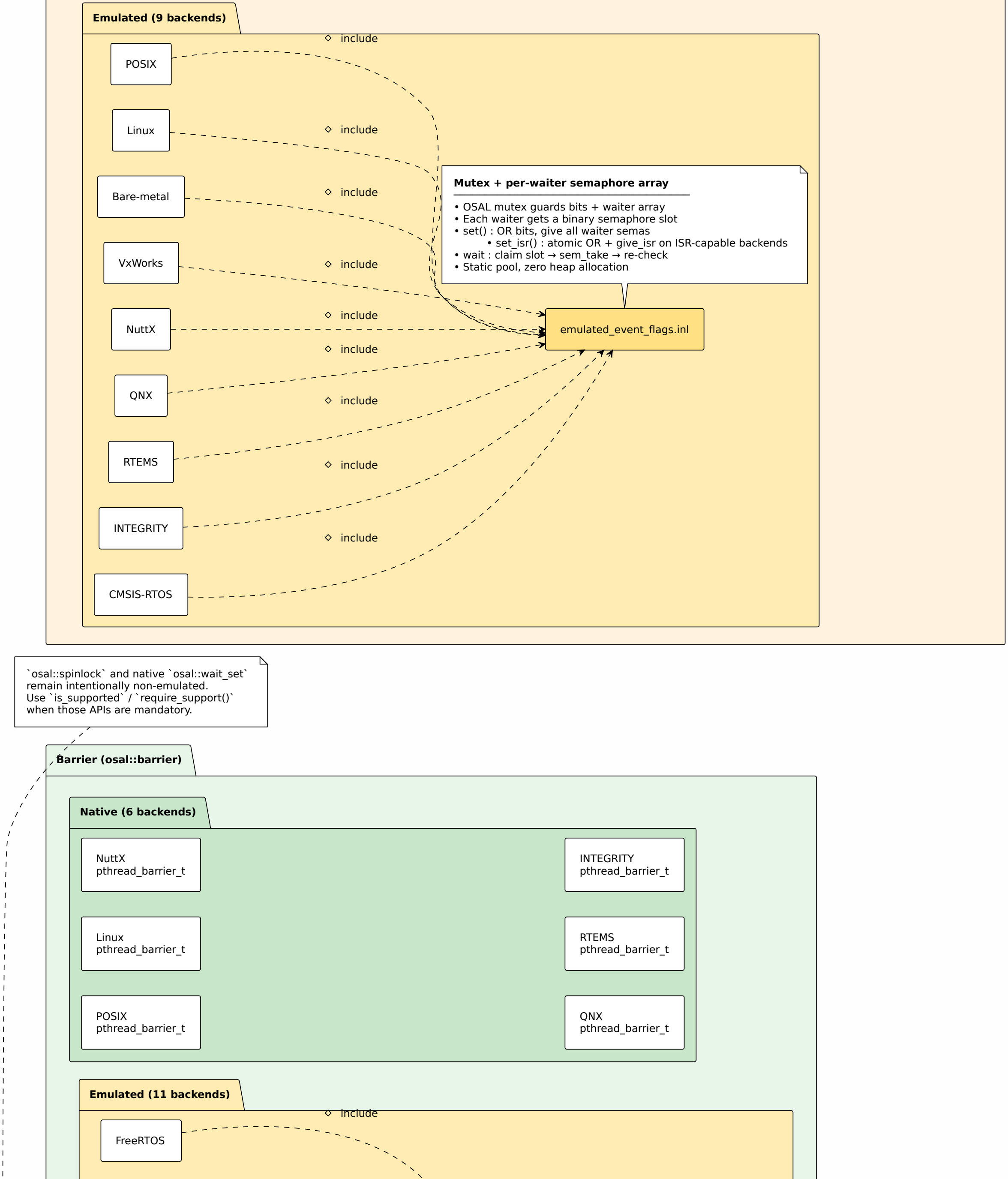
Polling multiplexer

- Polls non-destructive readiness queries on OSAL objects
- Supports queue, mailbox, event_flags, stream_buffer, message_buffer, work_queue, delayable_work, notification
- No backend ABI; pure C++

Portable composition

- osal::mutex + osal::condvar
- N indexed 32-bit slots
- set_bits / increment / overwrite / no_overwrite actions
- No new backend ABI required

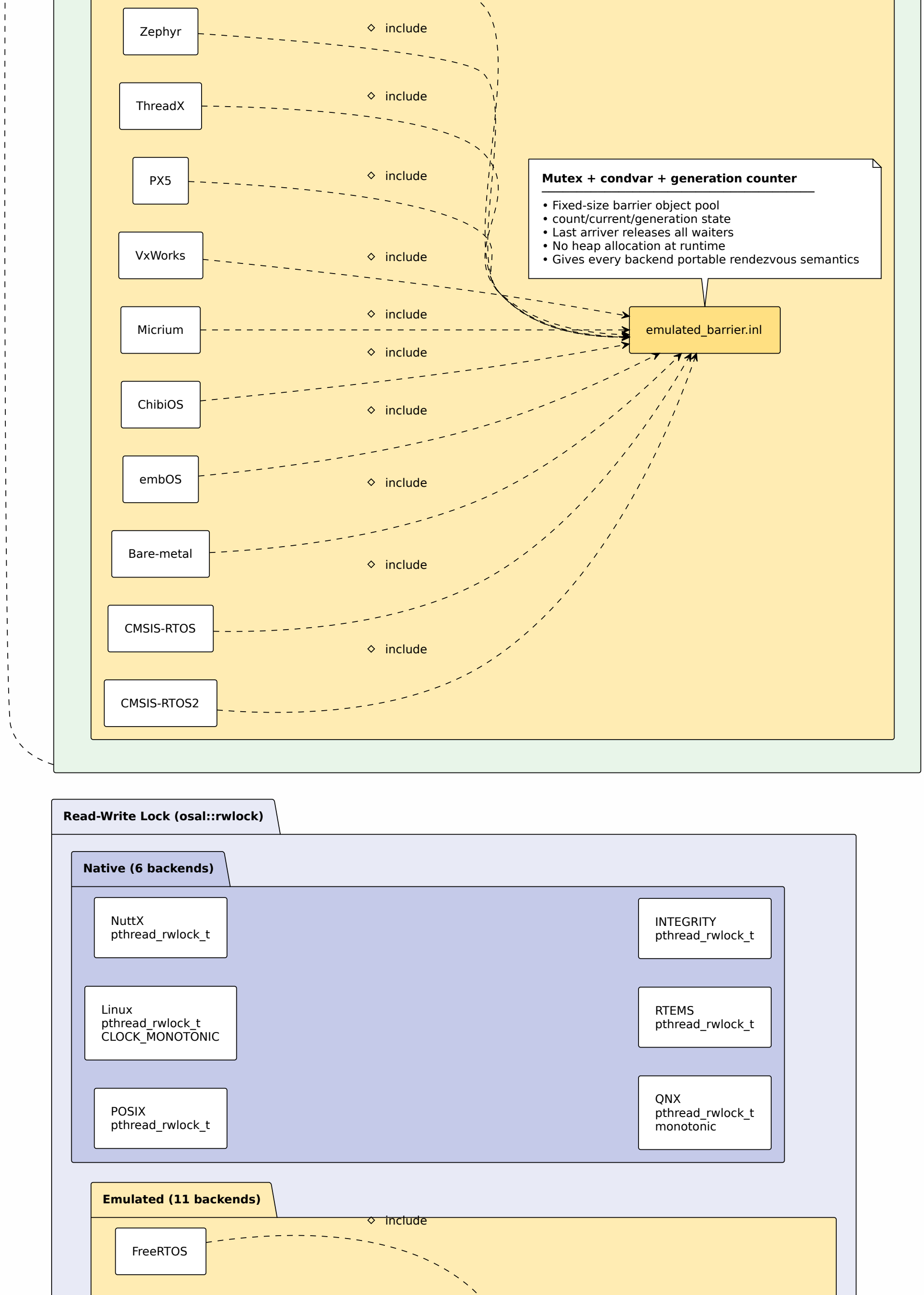

Linux
include
QNX
include
NuttX
include
RTEMS
include
INTEGRITY
include
VxWorks
include
Micrium
include
ChibiOS
include
embOS
include
Bare-metal
include
CMSIS-RTOS
include
CMSIS-RTOS2
SPSC ring + 2 binary semaphores
• Byte-oriented bulk read/write
• Two-chunk memcpy across wrap
• data_sem: signals bytes available
• space_sem: signals space available
• TriggerLevel: min bytes before wake
• head_ stored once per write (atomic)
• ISR path: osal_semaphore_give_isr
emulated_stream_buffer.inl
Event Flags (osal::event_flags)
Native (8 backends)
ThreadX tx_event_flags
ChibiOS event_source_t
Zephyr k_event
Micrium OSFlagCreate
CMSIS-RTOS2 osEventFlagsNew
FreeRTOS xEventGroupCreate
PX5 px5_event_flags
embOS OS_EVENT

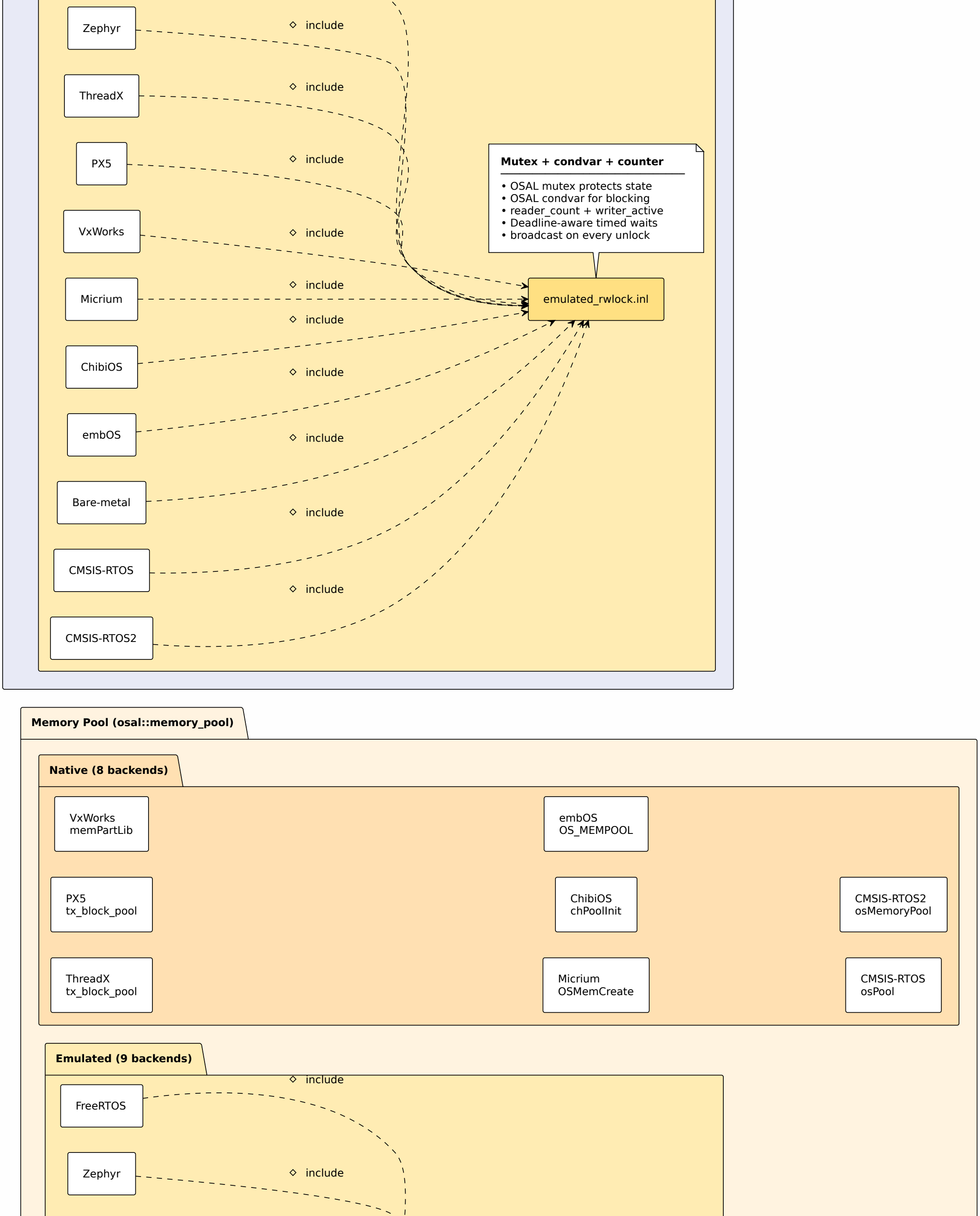

Emulated (9 backends)
POSIX
Linux
Bare-metal
VxWorks
NuttX
QNX
RTEMS
INTEGRITY
CMSIS-RTOS
include
include
include
include
include
include
include
include
include
emulated_event_flags.inl
Barrier (osal::barrier)
Native (6 backends)
NuttX pthread_barrier_t
Linux pthread_barrier_t
POSIX pthread_barrier_t
INTEGRITY pthread_barrier_t
RTEMS pthread_barrier_t
QNX pthread_barrier_t
Emulated (11 backends)
include
FreeRTOS

Mutex + per-waiter semaphore array

- OSAL mutex guards bits + waiter array
- Each waiter gets a binary semaphore slot
- set() : OR bits, give all waiter semas
  - set_isr() : atomic OR + give_isr on ISR-capable backends
- wait : claim slot → sem_take → re-check
- Static pool, zero heap allocation

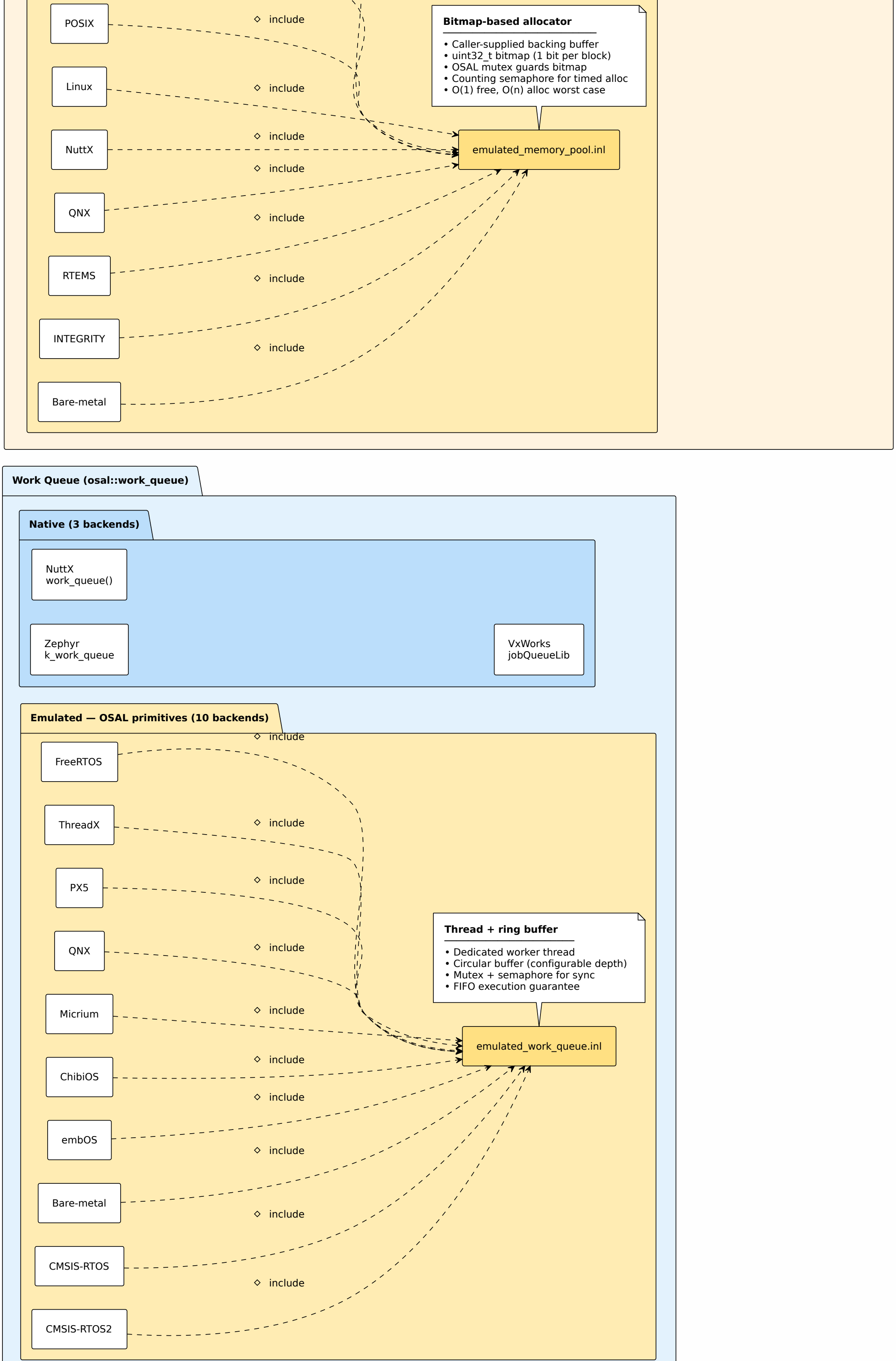
`osal::spinlock` and native `osal::wait_set`
remain intentionally non-emulated.
Use `is_supported` / `require_support()`
when those APIs are mandatory.

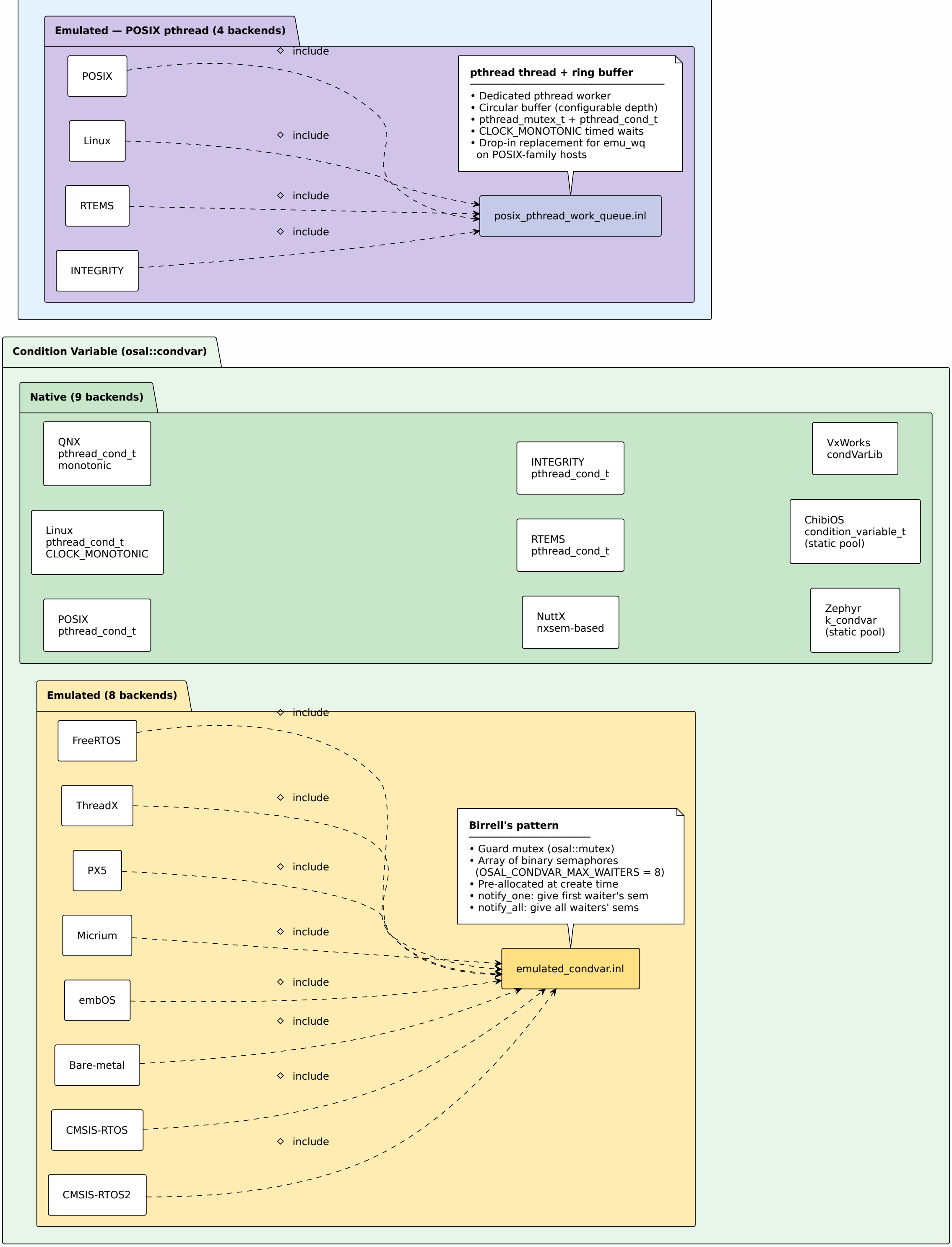

Zephyr
include
ThreadX
include
PX5
include
Mutex + condvar + generation counter
• Fixed-size barrier object pool
• count/current/generation state
• Last arriver releases all waiters
• No heap allocation at runtime
• Gives every backend portable rendezvous semantics
VxWorks
include
Micrium
include
emulated_barrier.inl
include
ChibiOS
include
embOS
include
Bare-metal
include
CMSIS-RTOS
include
CMSIS-RTOS2
Read-Write Lock (osal::rwlock)
Native (6 backends)
NuttX
pthread_rwlock_t
INTEGRITY
pthread_rwlock_t
Linux
pthread_rwlock_t
CLOCK_MONOTONIC
RTEMS
pthread_rwlock_t
POSIX
pthread_rwlock_t
QNX
pthread_rwlock_t
monotonic
Emulated (11 backends)
include
FreeRTOS


Zephyr
ThreadX
PX5
VxWorks
Micrium
ChibiOS
embOS
Bare-metal
CMSIS-RTOS
CMSIS-RTOS2
include
emulated_rwlock.inl
Memory Pool (osal::memory_pool)
Native (8 backends)
VxWorks
memPartLib
embOS
OS_MEMPOOL
PX5
tx_block_pool
ChibiOS
chPoolInit
CMSIS-RTOS2
osMemoryPool
ThreadX
tx_block_pool
Micrium
OSMemCreate
CMSIS-RTOS
osPool
Emulated (9 backends)
FreeRTOS
Zephyr
include

Mutex + condvar + counter

- OSAL mutex protects state
- OSAL condvar for blocking
- reader_count + writer_active
- Deadline-aware timed waits
- broadcast on every unlock


POSIX
Linux
NuttX
QNX
RTEMS
INTEGRITY
Bare-metal
include
include
include
include
include
include
include
Bitmap-based allocator
• Caller-supplied backing buffer
• uint32_t bitmap (1 bit per block)
• OSAL mutex guards bitmap
• Counting semaphore for timed alloc
• O(1) free, O(n) alloc worst case
emulated_memory_pool.inl
Work Queue (osal::work_queue)
Native (3 backends)
NuttX work_queue()
Zephyr k_work_queue
VxWorks jobQueueLib
Emulated — OSAL primitives (10 backends)
FreeRTOS
ThreadX
PX5
QNX
Micrium
ChibiOS
embOS
Bare-metal
CMSIS-RTOS
CMSIS-RTOS2
include
include
include
include
include
include
include
include
include
include
Thread + ring buffer
• Dedicated worker thread
• Circular buffer (configurable depth)
• Mutex + semaphore for sync
• FIFO execution guarantee
emulated_work_queue.inl

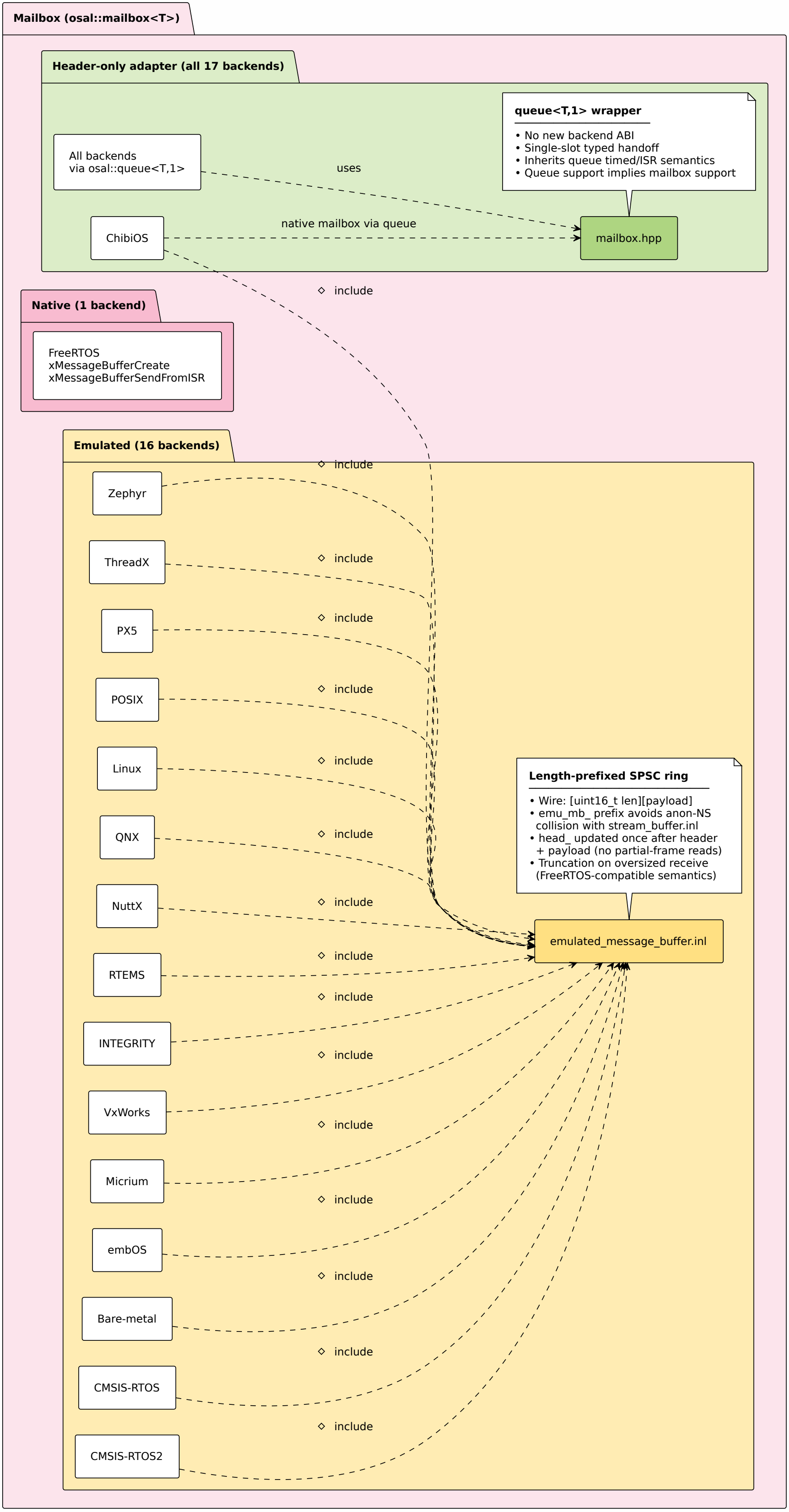

Emulated — POSIX pthread (4 backends)
POSIX
Linux
RTEMS
INTEGRITY
include
include
include
include
posix_pthread_work_queue.inl
Condition Variable (osal::condvar)
Native (9 backends)
QNX pthread_cond_t monotonic
Linux pthread_cond_t CLOCK_MONOTONIC
POSIX pthread_cond_t
INTEGRITY pthread_cond_t
RTEMS pthread_cond_t
NuttX nxsem-based
VxWorks condVarLib
ChibiOS condition_variable_t (static pool)
Zephyr k_condvar (static pool)
Emulated (8 backends)
FreeRTOS
ThreadX
PX5
Micrium
embOS
Bare-metal
CMSIS-RTOS
CMSIS-RTOS2
include
include
include
include
include
include
include
include
emulated_condvar.inl

pthread thread + ring buffer

- Dedicated pthread worker
- Circular buffer (configurable depth)
- pthread_mutex_t + pthread_cond_t
- CLOCK_MONOTONIC timed waits
- Drop-in replacement for emu_wq on POSIX-family hosts

Birrell's pattern

- Guard mutex (osal::mutex)
- Array of binary semaphores (OSAL_CONDVAR_MAX_WAITERS = 8)
- Pre-allocated at create time
- notify_one: give first waiter's sem
- notify_all: give all waiters' sems


Mailbox (osal::mailbox<T>)
Header-only adapter (all 17 backends)
All backends
via osal::queue<T,1>
uses
ChibiOS
native mailbox via queue
mailbox.hpp
include
Native (1 backend)
FreeRTOS
xMessageBufferCreate
xMessageBufferSendFromISR
Emulated (16 backends)
Zephyr
ThreadX
PX5
POSIX
Linux
QNX
NuttX
RTEMS
INTEGRITY
VxWorks
Micrium
embOS
Bare-metal
CMSIS-RTOS
CMSIS-RTOS2
emulated_message_buffer.inl

queue<T,1> wrapper

- No new backend ABI
- Single-slot typed handoff
- Inherits queue timed/ISR semantics
- Queue support implies mailbox support

Length-prefixed SPSC ring

- Wire: [uint16_t len][payload]
- emu_mb_ prefix avoids anon-NS collision with stream_buffer.inl
- head_ updated once after header + payload (no partial-frame reads)
- Truncation on oversized receive (FreeRTOS-compatible semantics)